IAM & Service Account Guide

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This page defines how services on the platform (api-service, auth-service, worker-service, ingestion-service, gateway, scheduler) authenticate to Google Cloud APIs and to each other, and how we grant, scope, and protect their identities. The guiding rules: use keyless credentials, grant least privilege, and never let a long-lived secret leave a secret store. Following this prevents the two most common auth failures — missing credentials and missing permissions — and eliminates the standing risk of leaked keys.

Standards

Keyless authentication with ADC

Every service authenticates with Application Default Credentials (ADC). Locally this resolves to the developer's identity; when deployed it resolves to the runtime service account — no key files. Attach a token via the standard auth library rather than hand-managing credentials, and let the transport refresh it.

```
import google.auth
from google.auth.transport.requests import AuthorizedSession

credentials, project = google.auth.default(
   scopes=["https://www.googleapis.com/auth/cloud-platform"])
session = AuthorizedSession(credentials)  # auto-refreshes; never cache tokens past expiry
resp = session.get("https://REGION-aiplatform.googleapis.com/v1/...")
```

- Give each service its own runtime service account — never share one identity across services.
- Use Workload Identity (GKE) or the built-in runtime service account (Cloud Run); do not download JSON keys.

Least-privilege role grants

Grant the narrowest predefined role that satisfies the call, on the specific target resource, then redeploy. Verify the grant before shipping instead of discovering it in production.

```
# Grant only what the service needs, scoped to the resource
gcloud projects add-iam-policy-binding PROJECT_ID \
 --member="serviceAccount:api-service@PROJECT_ID.iam.gserviceaccount.com" \
 --role="roles/aiplatform.user"

# Confirm the permission before relying on it
gcloud projects test-iam-permissions PROJECT_ID \
 --permissions=aiplatform.endpoints.predict
```

- Prefer predefined least-privilege roles (e.g. `roles/aiplatform.user`) over broad primitive roles like Editor/Owner.
- Scope grants to the resource (dataset, bucket, topic), not the whole project, where possible.

Secret handling

- Never commit credentials. Any unavoidable secret lives in Secret Manager and is mounted or fetched at runtime.
- Enable secret scanning on the repository so a leaked credential is caught at push time.
- Rotate and revoke on any suspicion of exposure; treat a leaked key as compromised immediately.

Common failures and fixes

- Deployed service gets 403 PERMISSION_DENIED even with credentials attached → grant the runtime service account the least-privilege role it needs on the target resource, redeploy, and verify with `gcloud ... test-iam-permissions` (BUG-1017).
- A JSON service account key was committed and flagged by secret scanning → revoke the key immediately, purge it from history, switch to keyless auth (ADC / Workload Identity), and move any real secret to Secret Manager (BUG-1021).
- Calls to a Google Cloud API return 401 from a container with no credentials attached → use ADC via `google.auth.default()` so the runtime service account's token is attached automatically (BUG-1006).

Checklist

- Service uses ADC; no JSON key files are downloaded or mounted.
- Each service has its own runtime service account (Workload Identity on GKE).
- Only least-privilege predefined roles are granted, scoped to the target resource.
- IAM grants verified with `test-iam-permissions` before deploy.
- Secret scanning enabled; secrets stored only in Secret Manager.
- Leaked or unused keys are revoked and rotated promptly.

Related incidents

BUG-1017, BUG-1021, BUG-1006